

NAME	KEN KIBUGI MUIRURI
TRACKING NUMBER	CS-SA12-26014
COHORT	CYBERSHUJAA COHORT 1-2026
TASK	INTRODUCTION TO WEB APPLICATIONS MODULE REPORT

Contents

1.0 Introduction to Web Applications	3
1.1 Introduction	3
1.2 Web Application Layout	3
1.3 Front End vs. Back End.....	3
2.0 Front End Components.....	3
2.1 HTML.....	3
2.2 Cascading Style Sheets (CSS)	4
2.3 JavaScript	5
3.0 Front End Vulnerabilities	5
3.1 Sensitive Data Exposure	5
3.2 HTML Injection.....	6
3.3 Cross-Site Scripting (XSS)	7
3.4 Cross-Site Request Forgery (CSRF)	8
4.0 Back End Components.....	8
4.1 Back End Servers.....	8
4.2 Web Servers.....	8
4.3 Databases	9
4.4 Development Frameworks & APIs	10
5.0 Back End Vulnerabilities	11
5.1 Common Web Vulnerabilities.....	11
5.2 Public Vulnerabilities	12
6.0 Next Steps.....	13
6.1 Next Steps.....	13
Conclusion	14

INTRODUCTION TO WEB APPLICATIONS REPORT

Introduction

In this report, I explore the fundamentals of web applications, their structure, and common security vulnerabilities. While I'm already familiar with most aspects of web applications, this report focuses on consolidating my understanding and linking it to security concepts.

Here is a shareable link to my completed module:

<https://academy.hackthebox.com/achievement/2380391/75>

1.0 Introduction to Web Applications

1.1 Introduction

In this section, I learned what web applications are and how they differ from traditional static websites. I understood the client-server architecture, including the separation between front-end and back-end components, and how web applications provide dynamic, interactive functionality.

1.2 Web Application Layout

In this section, I learned how web applications are structured behind the scenes, including their infrastructure, components, and architecture. I understood common deployment models such as single-server, multi-server, and segmented database setups, as well as the three-tier architecture (presentation, application, and data layers).

1.3 Front End vs. Back End

I learned that the front end is the client-side part of a web application that users see and interact with in the browser using HTML, CSS, and JavaScript, while the back end runs on the server and handles core logic, databases, and request processing. I also learned that poor security or input handling on either side can lead to serious vulnerabilities such as injections, which is why understanding both areas is important for web application penetration testing.

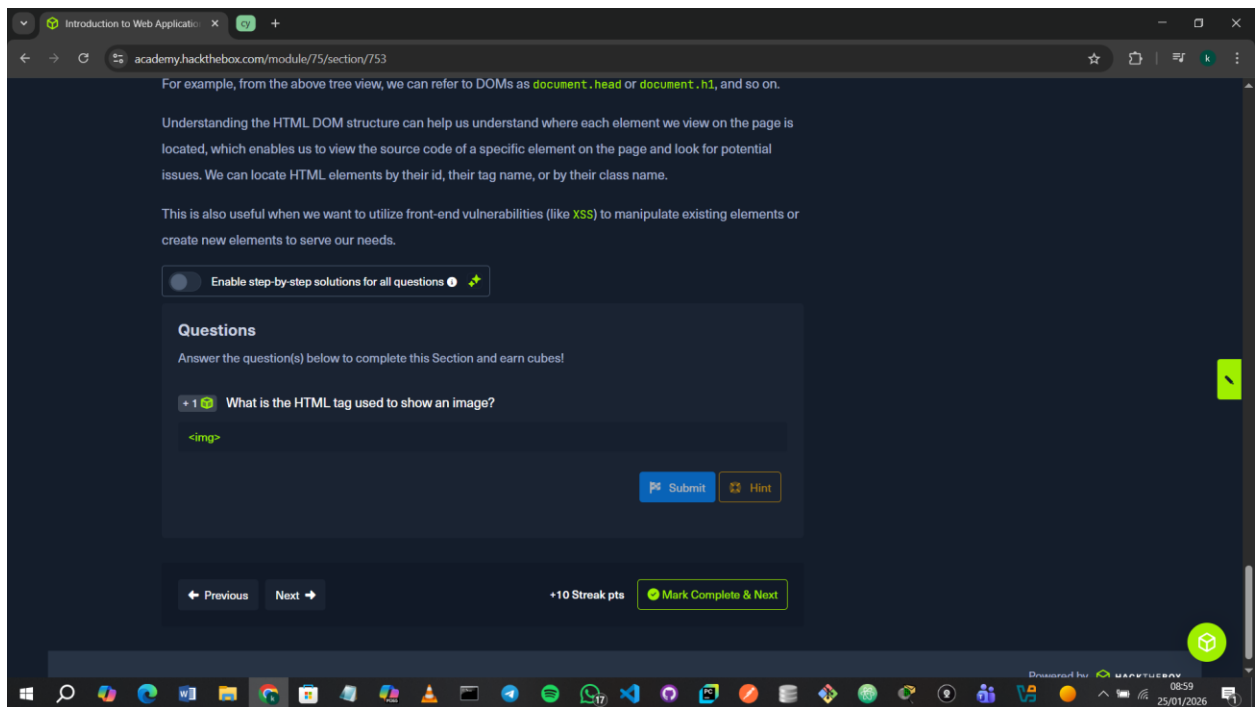
2.0 Front End Components

2.1 HTML

I learned that HTML is the core structure of web pages and defines elements such as headings, text, images, and forms, which browsers render for users. I also learned about the HTML DOM structure, URL encoding, and how understanding element hierarchy and attributes is important for analyzing front-end behavior and identifying vulnerabilities like XSS.

Question: What is the HTML tag used to show an image?

Answer: ****. I have worked with web applications a lot so I was familiar with this.

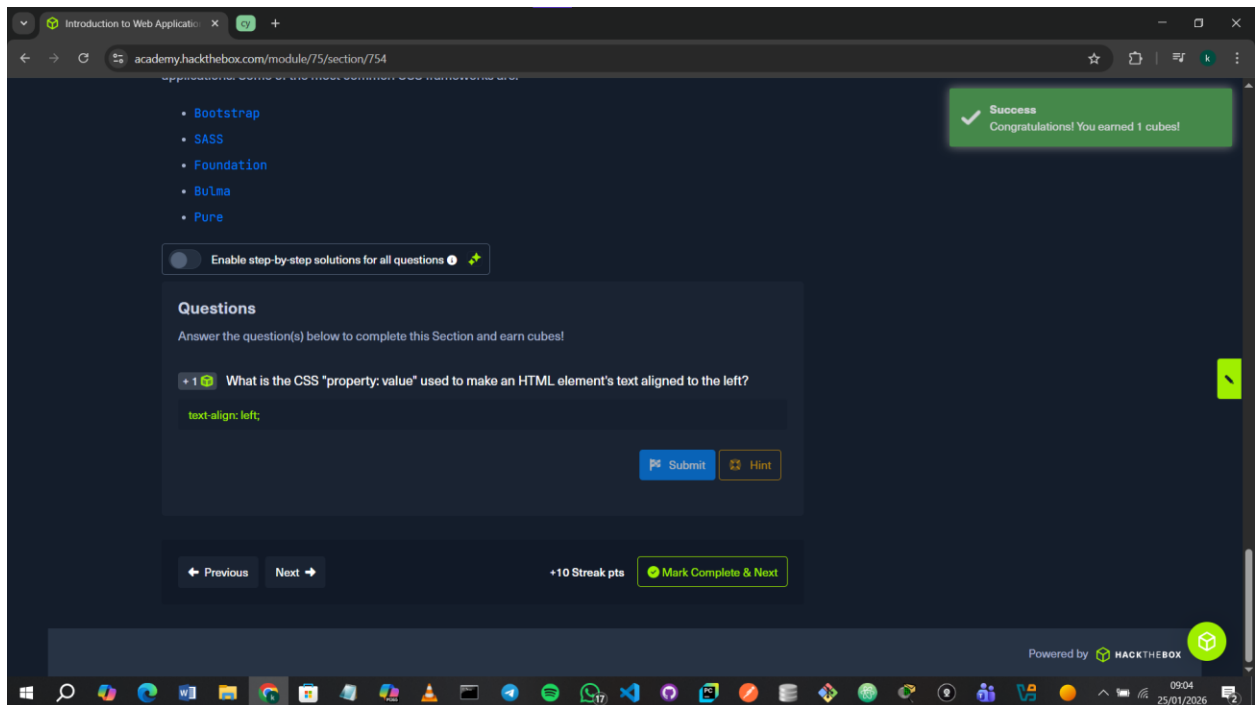


2.2 Cascading Style Sheets (CSS)

I learned that CSS is used to control the visual appearance of web pages by styling HTML elements, such as layout, colors, fonts, and animations.

Question: What is the CSS “property:value” used to make an HTML’s element’s text aligned to the left?

Answer: **text-align: left;**. I have also worked with CSS quite a lot.



2.3 JavaScript

I learned that JavaScript is responsible for adding functionality and interactivity to web pages. It allows web pages to update content dynamically, handle user input, communicate with the back end through HTTP requests, and drive advanced behaviors such as animations. I also learned that modern web applications rely heavily on JavaScript frameworks like React, Angular, and Vue to efficiently build dynamic and responsive interfaces.

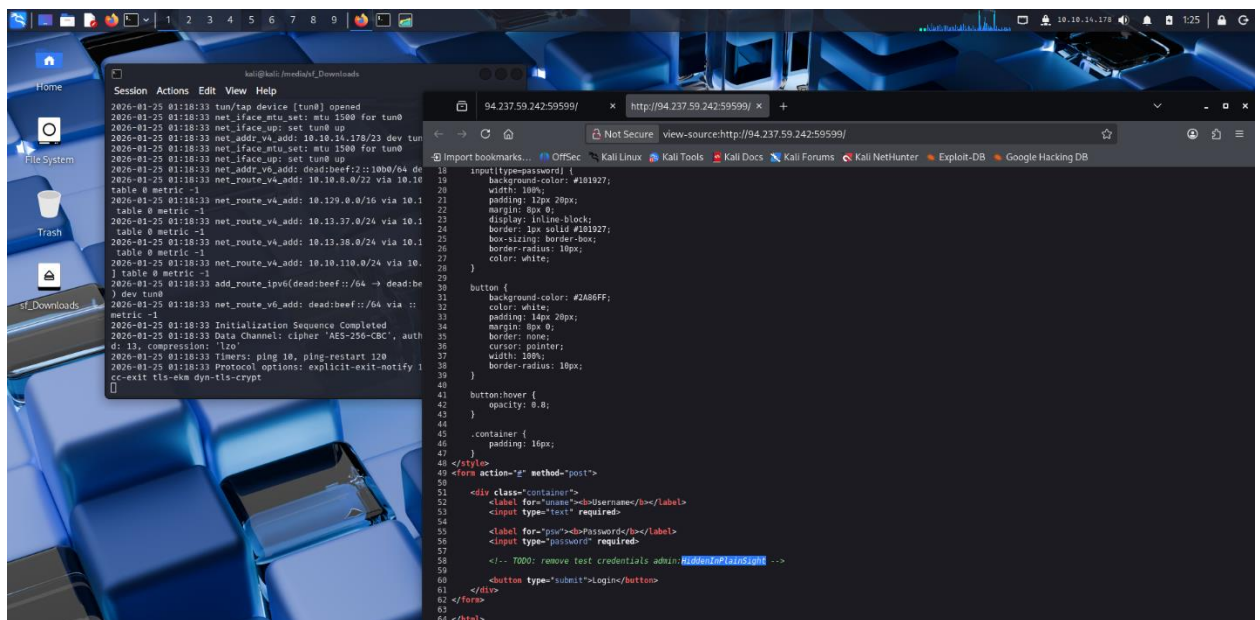
3.0 Front End Vulnerabilities

3.1 Sensitive Data Exposure

I learned that sensitive data exposure occurs when confidential information is unintentionally revealed in a web application's front-end source code. I learned how attackers can inspect page source to find exposed credentials, hidden links, or comments left by developers, and how this information can be leveraged to gain further access to the application or its back-end systems.

Question: Check the above login form for exposed passwords. Submit the password as the answer.

Answer: I opened the spawned address and viewed the source code for the page. The exposed password was **HiddenInPlainSight**.

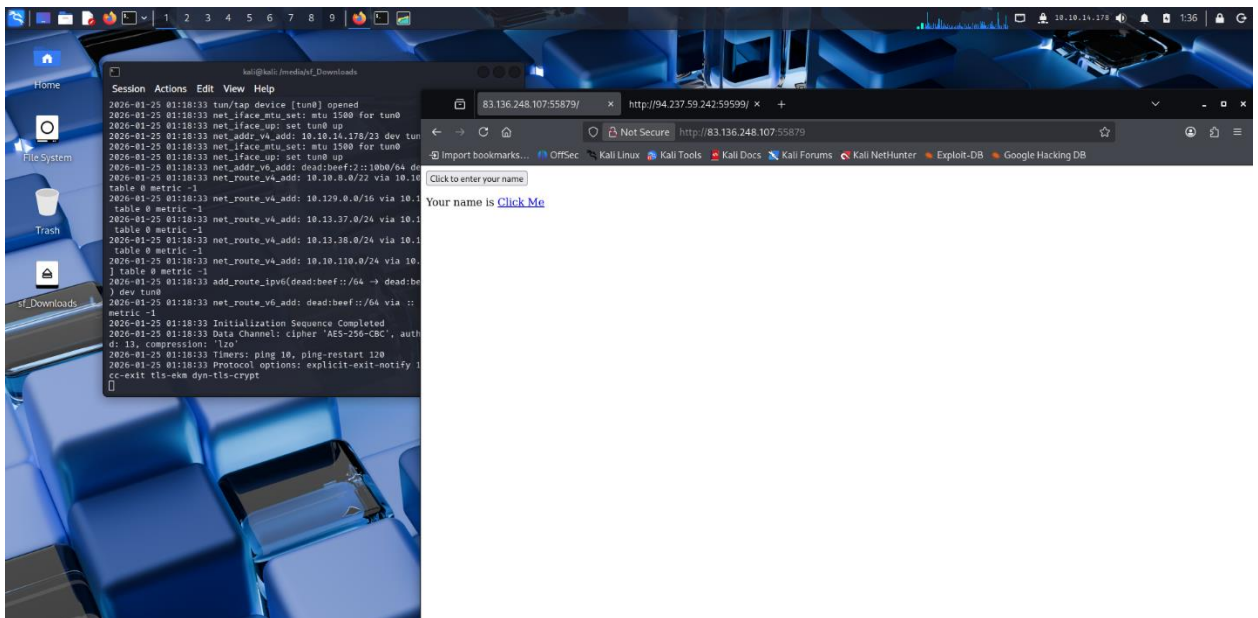


3.2 HTML Injection

I learned that HTML injection occurs when user input is not properly validated or sanitized and is rendered directly on a web page. I learned how attackers can inject malicious HTML to alter page content, deface websites, or trick users into submitting sensitive information through fake forms, which can lead to security and reputational damage.

Question: What text would be displayed on the page if we use the following payload as our input: `<a href="http://www.hackthebox.com">Click Me</a>`

Answer: **Your name is Click Me**

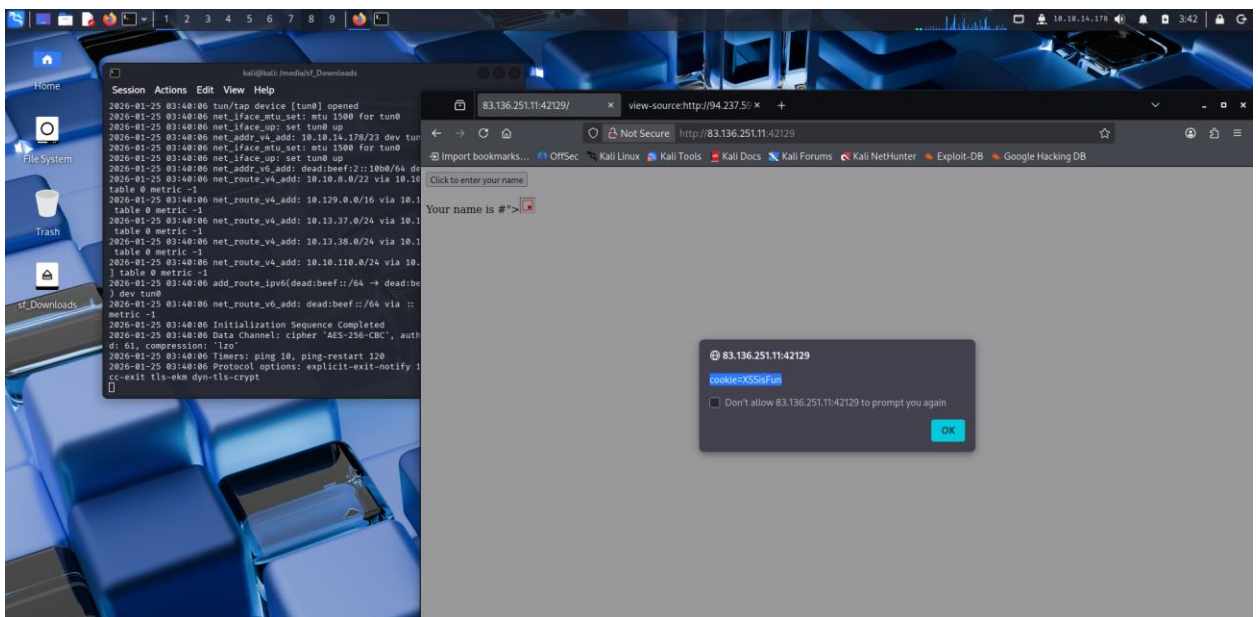


3.3 Cross-Site Scripting (XSS)

I learned that XSS occurs when attackers inject malicious JavaScript into a web page, allowing code to execute on the client-side.

Question: Try to use XSS to get the cookie value in the above page.

Answer: I used the JS code `"><img src=/ onerror=alert(document.cookie)>` as input and found the cookie value as **XSSisFun**.



3.4 Cross-Site Request Forgery (CSRF)

I learned that CSRF attacks exploit a user's authenticated session to perform unauthorized actions on a web application, such as changing account details, without the user's knowledge.

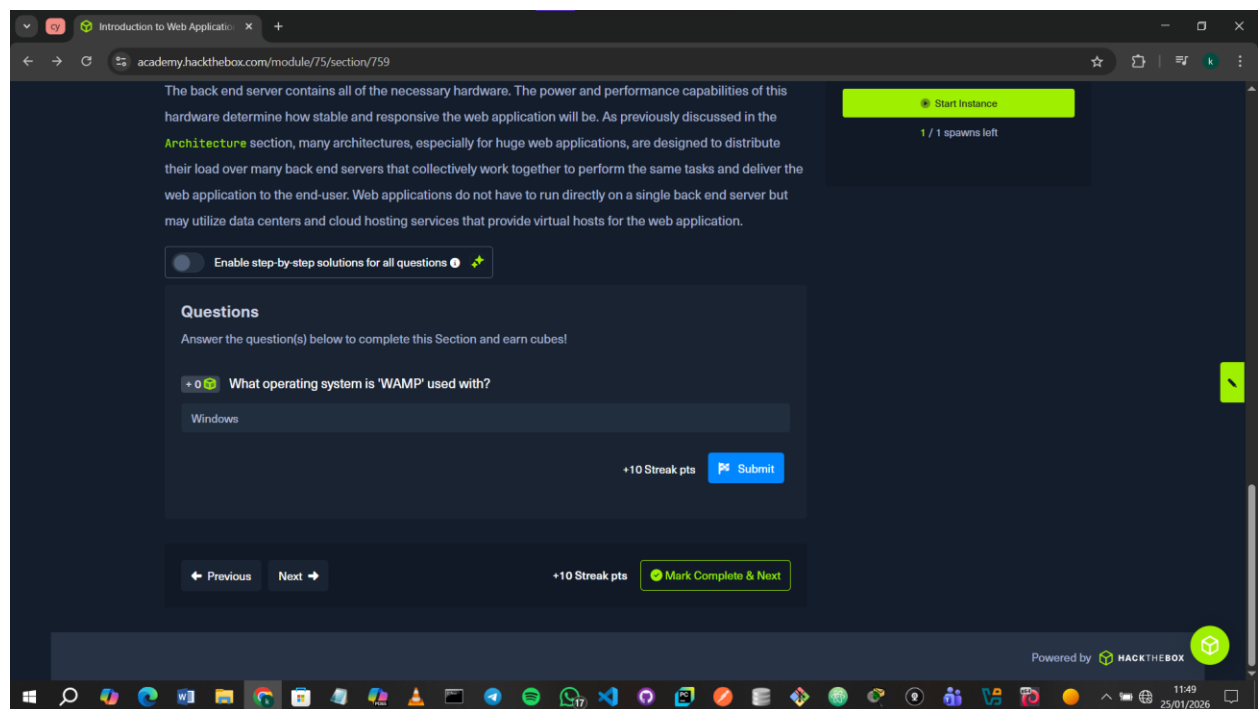
4.0 Back End Components

4.1 Back End Servers

I learned that back end servers are the physical or virtual systems that run the operating system and host all core web application components, including the web server, database, and development frameworks. I also understood how different technology stacks (such as LAMP and WAMP) are used and how server hardware, virtualization, and cloud infrastructure affect performance, scalability, and reliability.

Question: What operating system is "WAMP" used with?

Answer: **Windows**



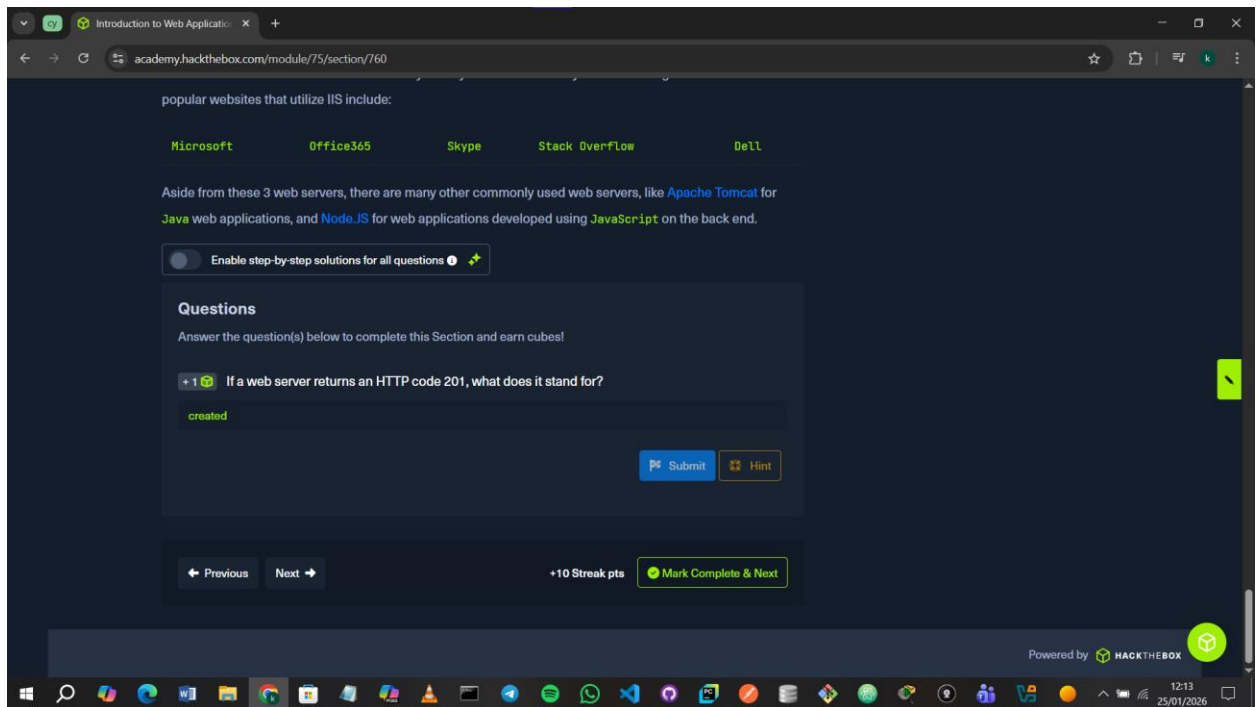
4.2 Web Servers

I learned that web servers handle HTTP requests from clients, route them to the correct web application resources, and return appropriate responses using standard HTTP status codes. I also learned the roles and differences between common web servers

such as Apache, NGINX, and IIS, and how their architecture and performance characteristics affect how web applications handle traffic.

Question: If a web server returns an HTTP code 201, what does it stand for?

Answer: Created



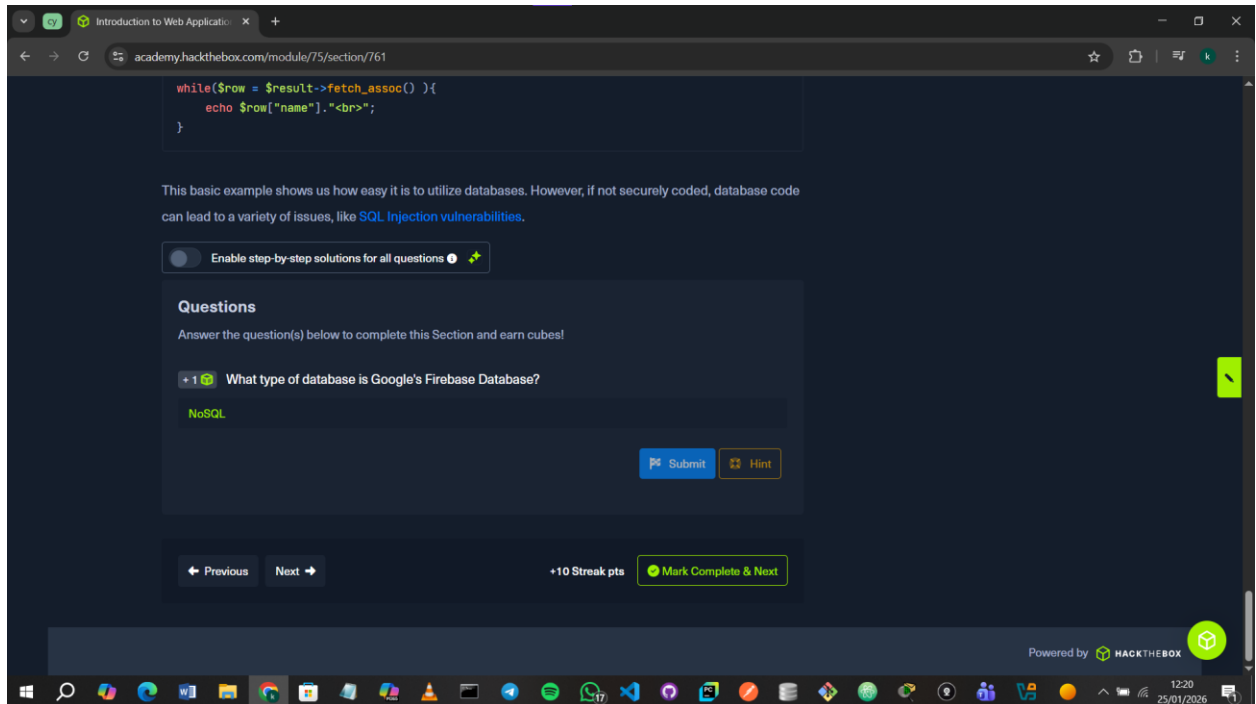
4.3 Databases

I learned that databases are a core back end component used by web applications to store and retrieve data such as user information, application content, and files. Databases enable dynamic web content by allowing applications to efficiently manage data that changes per user. Developers choose databases based on factors such as speed, scalability, structure, and cost.

Relational (SQL) databases store data in structured tables with defined schemas and relationships between tables, making them ideal for large, well-organized datasets. Common SQL databases include MySQL, MSSQL, Oracle, and PostgreSQL. Non-relational (NoSQL) databases do not rely on fixed schemas and instead use flexible data models such as key-value or document-based storage, making them highly scalable and suitable for unstructured data. Popular NoSQL databases include MongoDB, Elasticsearch, and Apache Cassandra. Both database types are widely used in modern web applications and are integrated into back end servers to support efficient data storage and retrieval.

Question: What type of database is Google's Firebase Database?

Answer: NoSQL

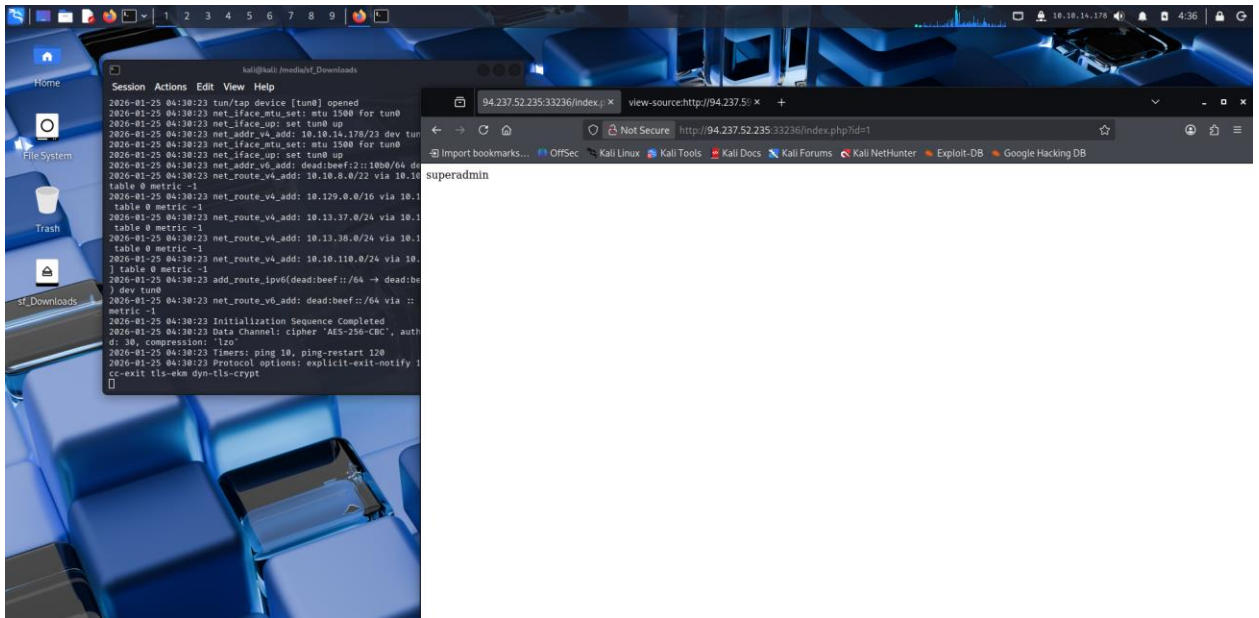


4.4 Development Frameworks & APIs

I learned that web frameworks like Django, Laravel, Express, and Rails make building complex web apps faster by providing common functionality. I also learnt that APIs connect the front end and back end, letting the front end send requests and get responses. Web APIs usually return data in JSON or XML. I learned that SOAP uses XML for complex data but is harder to use, while REST is simpler, uses URL paths, and is more common in modern apps.

Question: Use GET request '/index.php?id=0' to search for the name of the user with id number 1?

Answer: **superadmin**



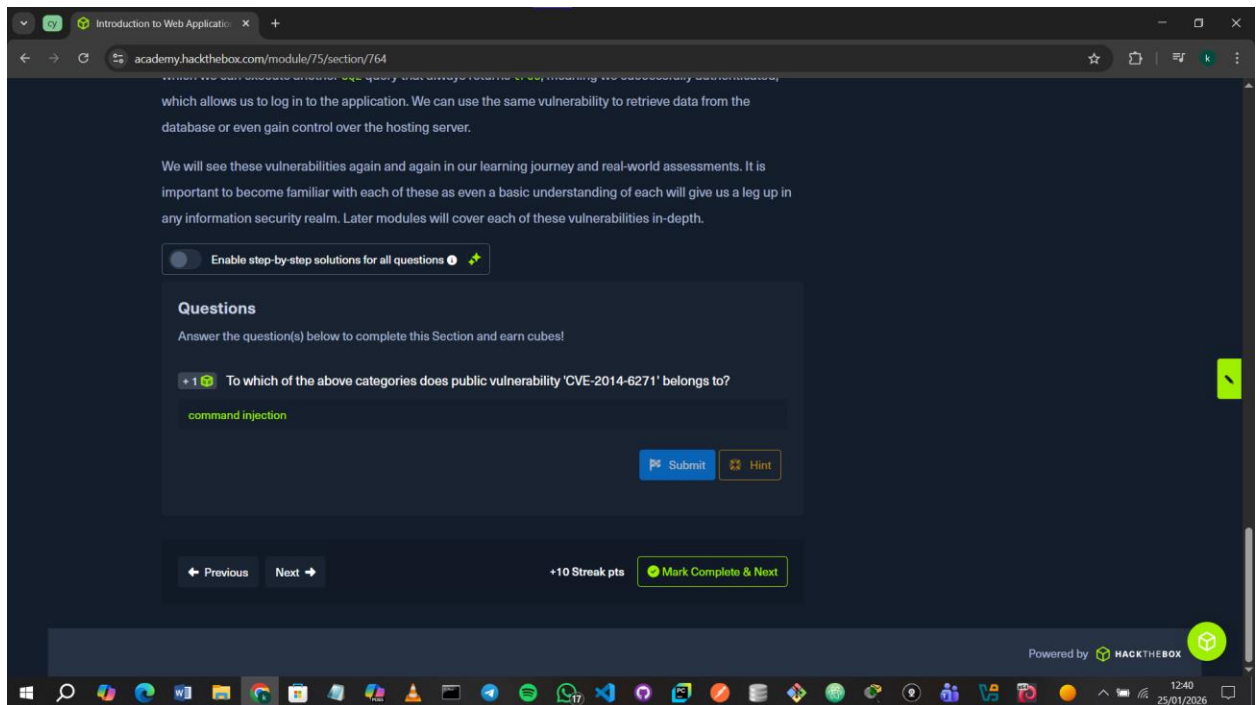
5.0 Back End Vulnerabilities

5.1 Common Web Vulnerabilities

I learnt that many web apps have vulnerabilities from misconfigurations or bad coding. Broken Authentication/Access **Control** lets attackers bypass login or access restricted areas. Malicious File Uploads allow harmful scripts to run if files aren't checked. Command Injection happens when input isn't sanitized and attackers can run server commands. SQL Injection lets attackers manipulate database queries to steal data or gain control.

Question: To which of the above concepts does public vulnerability 'CVE-2014-6271' belong to?

Answer: command injection

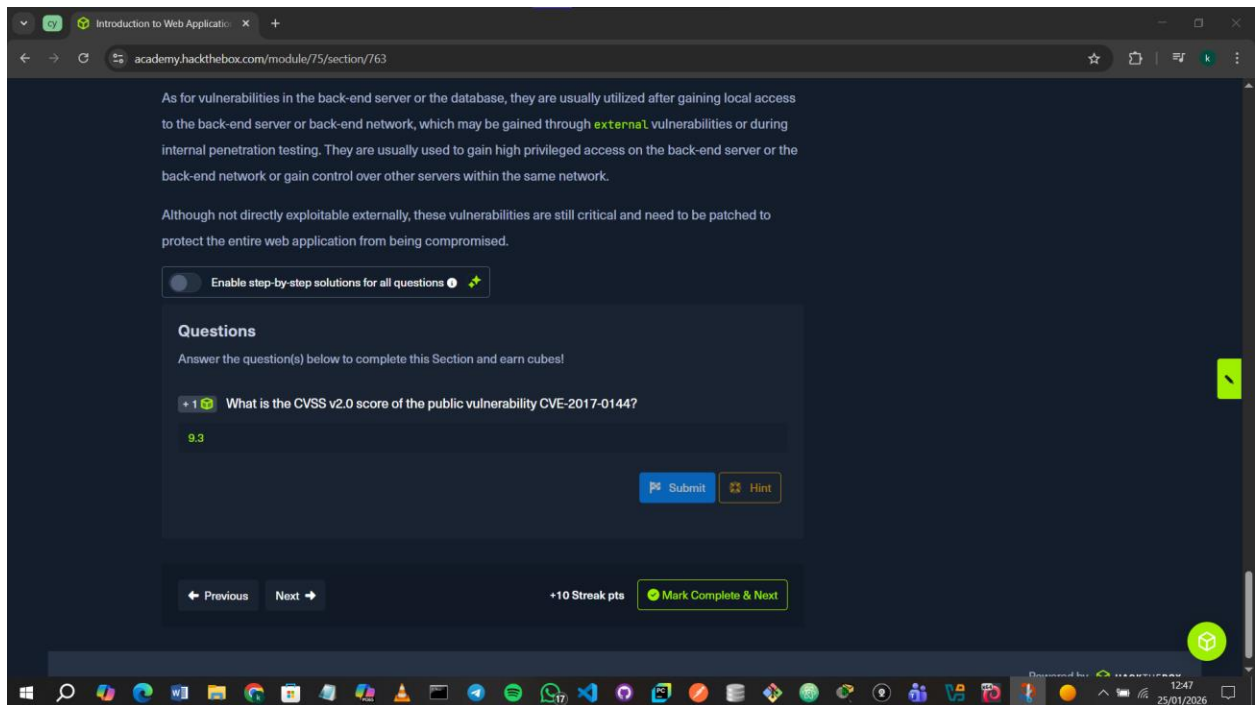


5.2 Public Vulnerabilities

I learnt that the most serious back-end vulnerabilities are the ones attackers can exploit externally to control the server. These usually come from coding mistakes in the back end. Many public web apps have known vulnerabilities assigned a CVE. I also learnt about CVSS, a system to score vulnerabilities from 0–10, helping prioritize which ones to fix first.

Question: What is the CVSS score of the public vulnerability CVE-2017-0144?

Answer: **9.3**

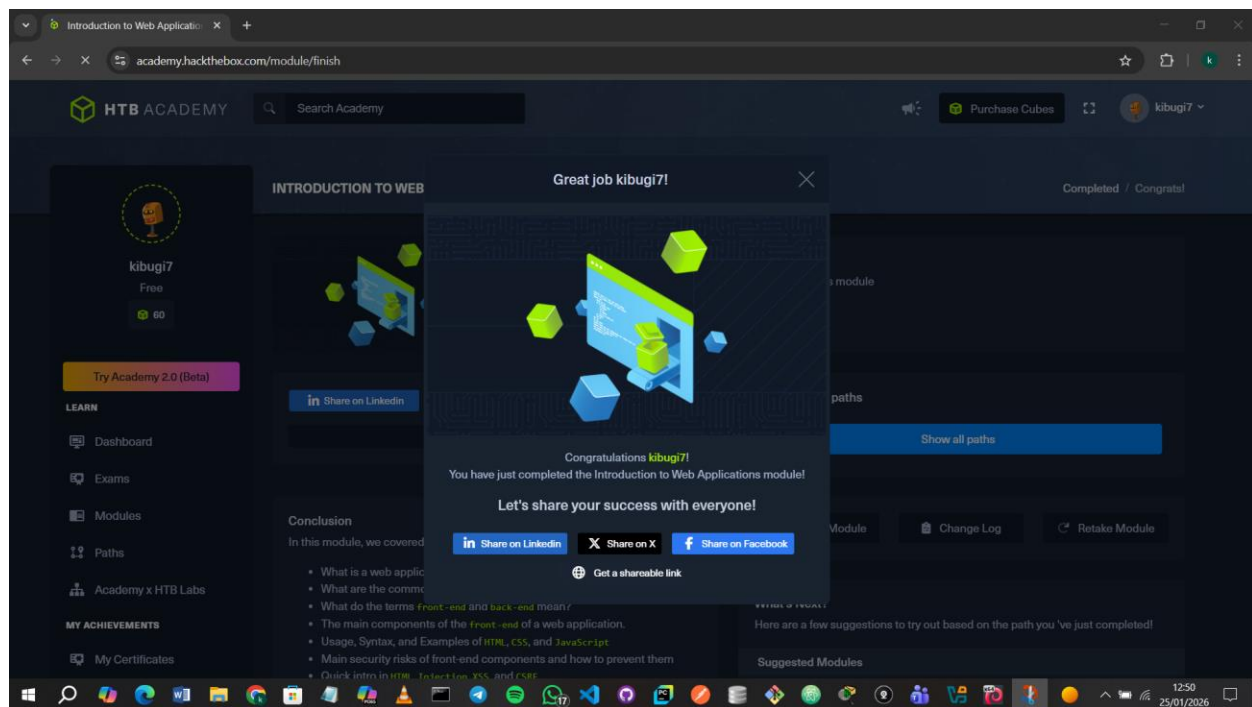


6.0 Next Steps

6.1 Next Steps

In this module, I learnt the basic structure of web applications, how they work, and the security risks they can introduce.

Below is a screenshot of my completed module:



Conclusion

In this module, I've reinforced my understanding of web applications and their components. Since I'm already familiar with most web app concepts, this module helped me consolidate my knowledge and connect it to security.